

Python上位机代码分析

本文档不讨论UI方面的内容，pyqt相关使用和信号类型请阅读教程内提供的博客，控件名称请自行查询

mainwindowlogic.py

此部分大多为UI上的操作，如修改和读取文本框内容，修改UI显示的内容，按钮的点击和信号传递（典型例子：ConnectButton控件点击动作绑定到connect_button_toggled_handler函数，若是连接网络，执行click_link_handler函数，进行一系列的文本框数据读取，最后当数据打包触发pyqtSignal类型的link_signal（此动作一是传递了得到的数据，二是类似点击按钮一样，点击了“link_signal”，因此在后面的main函数中，进一步将“点击link_signal”的动作绑定到新的函数link_signal_handler）

1. 导入模块

```
import time
import datetime
from PyQt5 import QtCore
from os import fstat

from PyQt5.QtCore import Qt, QTimer, pyqtSignal
from PyQt5.QtWidgets import QFileDialog, QMessageBox, QWidget, QMainWindow,
QGridLayout

# user file
from Network import get_host_ip, get_board_ip
import MainWindowUI
from UI.Mywidgets import PortInputDialog

# matplotlib
import matplotlib as mpl
mpl.use('Qt5Agg') # 声明使用 PyQt5
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAff as FigureCanvas
from matplotlib.backends.backend_qt5agg import NavigationToolbar2QT as
NavigationToolbar
import matplotlib.pyplot as plt
```

- **功能：**导入程序运行所需的模块，包括 PyQt5 的 GUI 组件、网络工具函数、Matplotlib 绘图库等。
- **说明：**
 - `PyQt5` 用于构建图形用户界面。
 - `matplotlib` 用于数据可视化。
 - 自定义模块 `Network` 和 `Mywidgets` 提供了网络工具和自定义控件。

2. 主类定义

```
class WidgetLogic(QMainWindow):
    link_signal = pyqtSignal(tuple) # 连接类型, 目标IP, 本机/目标端口, 总长度(字节数)
    disconnect_signal = pyqtSignal()
    counter_signal = pyqtSignal(int, int) # rx计数, tx计数
    fft_signal = pyqtSignal(str, str, str)
    send_signal = pyqtSignal(str)
    rx_num_signal = pyqtSignal(int)
```

- **功能:** 定义主窗口类 `WidgetLogic`, 继承自 `QMainWindow`。类中定义了多个信号, 用于在不同组件之间传递数据和状态信息。
- **说明:**
 - `link_signal`: 用于通知连接状态 (客户端或服务器)。
 - `disconnect_signal`: 用于通知断开连接。
 - `counter_signal`: 用于更新接收和发送数据的计数器。
 - `fft_signal`: 用于传递 FFT 分析结果。
 - `send_signal`: 用于发送数据。
 - `rx_num_signal`: 用于接收数据长度的验证。

3. 初始化方法

```
def __init__(self, parent=None):
    super().__init__(parent)
    # 变量定义
    self.SendCounter = 0
    self.file_name = None
    self.udp_per_rx = None
    self.figtoolbar1 = None
    self.canvas1 = None
    self.fig1 = None
    self.figtoolbar2 = None
    self.canvas2 = None
    self.fig2 = None
    self.protocol_type = "UDP" # 连接类型
    self.link_flag = self.NoLink # 连接状态flag
    self.receive_show_flag = True # 是否显示接收到的消息
    self.ReceiveCounter = 0 # 接收数据计数
    self.dir = None

    # MainwindowUI
    self.__ui = MainWindowUI.Ui_Mainwindow()
    self.__ui.setupUi(self)
    self.__ui.retranslateUi(self)

    # 输入框初始值设定
    self.__ui.MyHostAddrLineEdit.setText(get_host_ip()) # 显示本机IP地址
    self.__ui.RxPointLineEdit.setText(str(65536)) # 默认点数为65536
    self.__ui.fsLineEdit.setText(str(31250)) # 默认采样频率31250
    self.__ui.OSRLineEdit.setText(str(32)) # 默认过采样率32
```

```

self.__ui.lineEdit.setText(str(31)) # 默认采样周期数31

# 信号触发与槽函数连接
self.counter_signal.connect(self.counter_signal_handler)
self.__ui.pushButton_IP.clicked.connect(self.set_board_ip)
self.__ui.SaveButton.clicked.connect(self.save_data_init)
self.__ui.ConnectButton.toggled.connect(self.connect_button_toggled_handler)
self.__ui.SendButton.clicked.connect(self.send_link_handler)

self.__ui.CounterResetLabel.clicked.connect(self.counter_reset_button_handler)

self.__ui.ReceivePauseCheckBox.toggled.connect(self.receive_pause_checkbox_toggled_handler)
self.fft_signal.connect(self.fft_signal_handler)
self.rx_num_signal.connect(self.rx_num_signal_handler)

# 绘画界面UI初始化
self.plot_ui()

```

- **功能：**初始化主窗口的界面和逻辑。
- **说明：**
 - 初始化变量，设置默认值。
 - 加载 UI 文件并设置初始值。
 - 连接信号与槽函数，确保用户操作能够触发相应的逻辑。
 - 初始化绘图界面。

4. 连接与断开连接

```

def connect_button_toggled_handler(self, state):
    if state:
        self.click_link_handler()
    else:
        self.click_disconnect()
        self.editable(True)

def click_link_handler(self):
    # 连接逻辑
    # ...
    if self.protocol_type == "UDP" and server_flag:
        self.link_signal.emit((self.ServerUDP, my_ip, my_port, max_length, fs,
osr, fin))
        self.link_flag = self.ServerUDP
        self.__ui.StateLabel.setText('UDP服务端')
    elif self.protocol_type == "UDP" and not server_flag:
        self.link_signal.emit((self.ClientUDP, target_ip, target_port, "", "",
"", ""))
        self.link_flag = self.ClientUDP
        self.__ui.StateLabel.setText("UDP客户端")

def click_disconnect(self):
    self.disconnect_signal.emit()
    self.link_flag = self.NoLink

```

```
self.__ui.StateLabel.setText("未连接")
```

- **功能：**处理连接和断开连接的逻辑。
- **说明：**
 - `connect_button_toggled_handler`：根据按钮状态触发连接或断开连接。
 - `click_link_handler`：根据用户输入的参数（如 IP、端口等）建立连接。
 - `click_disconnect`：断开当前连接并重置状态。

5. 数据发送

```
def send_link_handler(self):  
    if self.link_flag != self.NoLink:  
        send_msg = self.__ui.SendPlainTextEdit.toPlainText()  
        self.send_signal.emit(send_msg)
```

- **功能：**发送用户输入的数据。
- **说明：**
 - 检查是否已建立连接。
 - 获取用户在文本框中输入的数据并发送。

6. 数据接收与显示

```
def msg_write(self, msg: str):  
    if self.receive_show_flag:  
        self.__ui.ReceivePlainTextEdit.appendPlainText(msg)  
  
def info_write(self, info: str, mode: int, rx_num: int):  
    if self.receive_show_flag:  
        if mode == self.InfoRec:  
            self.__ui.ReceivePlainTextEdit.appendHtml(f'<font color="blue">{info}</font>')  
            self.ReceiveCounter += rx_num  
            self.counter_signal.emit(self.ReceiveCounter, self.SendCounter)  
        elif mode == self.InfoSend:  
            self.__ui.ReceivePlainTextEdit.appendHtml(f'<font color="green">{info}</font>')
```

- **功能：**接收并显示数据。
- **说明：**
 - `msg_write`：将消息写入接收文本框。
 - `info_write`：根据数据类型（接收/发送）以不同颜色显示。

7. 数据计数与重置

```
def counter_signal_handler(self, receive_count, send_count):
    self.__ui.ReceiveCounterLabel.setText(str(receive_count))
    self.__ui.SendCounterLabel.setText(str(send_count))

def counter_reset_button_handler(self):
    self.ReceiveCounter = 0
    self.SendCounter = 0
    self.counter_signal.emit(self.ReceiveCounter, self.SendCounter)
```

- **功能：**更新和重置数据计数器。
- **说明：**
 - `counter_signal_handler`：更新接收和发送计数器的显示。
 - `counter_reset_button_handler`：将计数器重置为零。

8. 绘图功能

```
def plot_ui(self):
    self.fig1 = plt.figure()
    self.canvas1 = FigureCanvas(self.fig1)
    self.figtoolbar1 = NavigationToolbar(self.canvas1, self)
    self.gridlayout1 = QGridLayout(self.__ui.groupBox1)
    self.gridlayout1.addWidget(self.canvas1)
    self.gridlayout1.addWidget(self.figtoolbar1)

    self.fig2 = plt.figure()
    self.canvas2 = FigureCanvas(self.fig2)
    self.figtoolbar2 = NavigationToolbar(self.canvas2, self)
    self.gridlayout2 = QGridLayout(self.__ui.groupBox2)
    self.gridlayout2.addWidget(self.canvas2)
    self.gridlayout2.addWidget(self.figtoolbar2)
```

- **功能：**初始化绘图界面。
- **说明：**
 - 使用 Matplotlib 创建绘图窗口。
 - 将绘图窗口嵌入到 PyQt5 的布局中。

9. FFT 分析结果处理

```
def fft_signal_handler(self, snr_result, enob_result, max_result):
    self.__ui.SNDRLabel.setText(str(snr_result))
    self.__ui.enoblabe1.setText(str(enob_result))
    self.__ui.maxlabe1.setText(str(max_result))
```

- **功能：**显示 FFT 分析结果。
- **说明：**
 - 更新 UI 中的标签以显示 SNDR、ENOB 和最大值。

10. 数据保存

```
def save_data_init(self):
    self.file_name = QFileDialog.getSaveFileName(self, "保存到txt", "./data.txt",
        "ALL(*, *);;txt文件(*.txt)", "txt文件(*.txt)") [0]

def save_data(self, str_data):
    if self.savedata_flag == 1:
        try:
            with open(self.file_name, mode="a") as f:
                now = datetime.datetime.now()
                time_str = now.strftime("%Y-%m-%d %H:%M:%S")
                f.write(time_str)
                f.write('\r\n')
                f.write('start of saving data\n')
                f.write('new high sndr:')
                f.write(str(self.maxSNDR))
                f.write('\r\n')
                f.write('here is data str\n')
                f.write(str_data)
                f.write('\r\n')
                f.write('end of saving data\n')
                self.savedata_flag = 0
        except FileNotFoundError:
            pass
```

- **功能：**保存数据到文件。
- **说明：**
 - `save_data_init`：通过文件对话框选择保存路径。
 - `save_data`：将数据写入文件，记录时间戳和数据内容。

下面给出实际使用时可能要修改的内容

- 1、自定义UI的控件名称
- 2、控件所绑定的函数，文本框的预设值，文本框读取的变量
- 3、如果修改了变量名，注意修改if判断语句和其他用到变量名处的内容（结合报错信息debug）
- 4、`click_link_handler`函数中自行增加或删减判断条件

mainv2.py

此为上位机程序主函数，主要内容为各pyqtSignal类型信号被触发时所执行的函数以及画图函数，注意画图函数只能写在main函数中

1. 程序初始化

```
class MainWindow(QWidget, NetworkLogic):
    def __init__(self, parent=None):
        super().__init__(parent)
        # 检测屏幕分辨率并调整窗口大小
        self.desktop = QApplication.desktop()
        self.screenRect = self.desktop.screenGeometry()
        self.screenheight = self.screenRect.height()
        self.screenwidth = self.screenRect.width()
        self.height = int(self.screenheight * 0.7)
        self.width = int(self.screenwidth * 0.7)
        self.resize(self.width, self.height)
```

- **功能：**初始化主窗口，调整窗口大小以适应屏幕分辨率。
- **说明：**继承了 `QWidget` 和 `NetworkLogic`，整合了 UI 逻辑和网络通信功能。

2. 信号与槽的连接

```
self.link_signal.connect(self.link_signal_handler)
self.disconnect_signal.connect(self.disconnect_signal_handler)
self.send_signal.connect(self.send_signal_handler)
self.udp_signal_write_msg.connect(self.msg_write)
self.udp_signal_write_info.connect(self.info_write)
self.udp_signal_len_info.connect(self.len_write)
self.udp_save_str_info.connect(self.save_data)
self.plot_signal_data_info.connect(self.data_plot)
```

- **功能：**将自定义信号连接到对应的槽函数，实现功能模块之间的交互。
- **说明：**通过信号与槽机制，实现网络通信、数据处理和 UI 更新的解耦。

3. UDP 通信

```
def link_signal_handler(self, signal):
    link_type, ip, port, data_len, self.input_fs, self.input_OS, self.input_fin = signal
    if link_type == self.ServerUDP:
        self.udp_server_start(ip, port, data_len)
    elif link_type == self.ClientUDP:
        self.udp_client_start(ip, port)
```

- **功能：**处理 UDP 服务器或客户端的启动逻辑。
- **说明：**根据信号类型启动 UDP 服务器或客户端，设置相关参数。

```
def send_signal_handler(self, msg: str):
    if self.link_flag == self.ClientUDP:
        self.udp_send(msg)
        self.SendCounter += 1
    self.counter_signal.emit(self.ReceiveCounter, self.SendCounter)
```

- **功能：**发送数据到目标地址。

- **说明：**在客户端模式下发送数据，并更新发送计数器。

4. FFT 分析与数据处理

```
def data_plot(self):
    plot_data = self.int_info
    data_dot = int(self.data_byte / 4)
    ydata = np.array(plot_data)
    ydata = np.resize(ydata, (data_dot,))
    self.calc_snr2(ydata, data_dot, self.input_fs, self.input_osr,
self.input_fin, 0)
```

- **功能：**对接收到的数据进行 FFT 分析。
- **说明：**将接收到的数据进行归一化处理，并调用 `calc_snr2` 函数进行 FFT 分析。

```
def calc_snr2(self, origin_data, nfft, fs, osr, fin_m, sig_mode):
    # FFT 分析逻辑
    # ...
    self.SNDR = snrdB
    if self.SNDR > self.maxSNDR and not math.isinf(self.SNDR):
        self.maxSNDR = self.SNDR
        self.savedata_flag = 1
    self.enob = enob
    self.fft_signal.emit(str(self.SNDR), str(self.enob), str(self.maxSNDR))
```

- **功能：**计算信噪比 (SNDR) 和有效位数 (ENOB)。
- **说明：**通过 FFT 分析计算信号的功率谱密度，进而计算 SNDR 和 ENOB，并更新 UI。

5. 数据可视化

```
# 数据图
self.fig1.clf()
ax1 = self.fig1.subplots()
ax1.plot(np.arange(nfft), get_data * 1.8 / 2048)
ax1.set_title('data PLOT')
ax1.set_ylabel('Voltage(V)')
ax1.grid(which='both', axis='both')

# FFT 频谱图
self.fig2.clf()
ax2 = self.fig2.subplots()
ax2.plot(x_label, ptotdB_norm)
ax2.set_xscale("log")
ax2.set_title('PSD PLOT')
ax2.set_ylabel('AMPLITUDE(dBFS)')
ax2.grid(which='both', axis='both')
```

- **功能：**绘制数据波形图和 FFT 频谱图。
- **说明：**使用 Matplotlib 的 `plot` 函数绘制数据波形图和频谱图，并设置图表的标题、坐标轴标签和网格。

6. 数据保存

```
def save_data(self, str_data):
    if self.savedata_flag == 1:
        try:
            with open(self.file_name, mode="a") as f:
                now = datetime.datetime.now()
                time_str = now.strftime("%Y-%m-%d %H:%M:%S")
                f.write(time_str)
                f.write('\r\n')
                f.write('start of saving data\n')
                f.write('new high sndr:')
                f.write(str(self.maxSNDR))
                f.write('\r\n')
                f.write('here is data str\n')
                f.write(str_data)
                f.write('\r\n')
                f.write('end of saving data\n')
                self.savedata_flag = 0
        except FileNotFoundError:
            pass
```

- **功能：**将数据保存到文件。
- **说明：**将接收到的数据和分析结果保存到指定文件中。

主程序入口

```
if __name__ == '__main__':
    QtCore.QCoreApplication.setAttribute(QtCore.Qt.AA_EnableHighDpiScaling)
    app = QApplication(sys.argv)
    mywin = Mainwindow()
    mywin.show()
    sys.exit(app.exec_())
```

- **功能：**启动 PyQt5 应用程序。
- **说明：**创建 `MainWindow` 实例并显示主窗口，启动事件循环。

下面给出实际使用时可能要修改的内容

- 1、各自定义的变量名，pyqtSignal和触发的函数
- 2、link_signal_handler内将传递过来的UI数据拆解为需要的变量
- 3、数据处理与画图部分请根据需求重写，注意保留画布刷新与画布重绘语句，详见代码注释
- 4、代码内留存了一些计时debug语句，因为需要确保数据接收时间（每接收到一定量的数据触发画图函数）>数据处理+画图所需时间，否则程序有卡死风险

Udp.py

此代码为网络连接部分，socket相关内容无需修改，实际使用时修改接收数据定义的变量，所需接收的长度（多久触发一次画图函数的判断）

触发画图函数的语句逻辑：plot_signal_data_info被emit，这个动作在main函数中连接到画图函数

主要修改内容应在udp_server_start的变量名和udp_server_concurrency的变量名及判定逻辑中

获取本机 IP 地址

```
def get_host_ip() -> str:
    """获取本机IP地址"""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
    finally:
        s.close()
    return ip
```

- **功能：**通过连接到外部服务器（如 Google 的 DNS 服务器）获取本机的 IP 地址。
- **说明：**使用 UDP 套接字连接到外部服务器，获取绑定的本地 IP 地址。

2. 获取开发板 IP 地址

```
def get_board_ip():
    hostname = socket.gethostname()
    board_ip = socket.gethostbyname(hostname)
    return board_ip
```

- **功能：**获取当前主机的 IP 地址。
- **说明：**通过主机名获取 IP 地址，适用于开发板或本地主机。

3. UDP 通信类

```
class UdpLogic:
    udp_signal_write_msg = pyqtSignal(str)
    udp_signal_write_info = pyqtSignal(str, int, int)
    plot_signal_data_info = pyqtSignal(int)
    udp_signal_len_info = pyqtSignal(int)
    udp_save_str_info = pyqtSignal(str)
```

- **功能：**定义了 UDP 通信类 `UdpLogic`，包含多个信号用于与 PyQt5 GUI 交互。
- **说明：**
 - `udp_signal_write_msg`：用于发送消息到 GUI。
 - `udp_signal_write_info`：用于发送接收或发送的数据信息。
 - `plot_signal_data_info`：用于发送处理后的数据以供绘图。
 - `udp_signal_len_info`：用于发送接收数据的长度。
 - `udp_save_str_info`：用于发送原始数据字符串。

4. UDP 服务器启动

```
def udp_server_start(self, ip: str, port: int, data_len: int) -> None:
    """
    开启UDP服务端
    """
    self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    self.udp_socket.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, 1024 * 1024)
    address = (ip, port)
    self.udp_socket.bind(address)
    self.sever_th = threading.Thread(target=self.udp_server_concurrency)
    self.sever_th.start()
    msg = "UDP服务端正在监听端口:{ }\n".format(port)
    self.udp_signal_write_msg.emit(msg)
```

- **功能：**启动 UDP 服务器，监听指定的 IP 和端口。
- **说明：**
 - 创建 UDP 套接字并绑定到指定地址。
 - 启动线程以并发处理数据接收。
 - 发送状态消息到 GUI。

5. UDP 服务器并发处理

```
def udp_server_concurrency(self) -> None:
    """
    用于创建一个线程持续监听UDP通信
    """
    while True:
        total_bytes_info = b''
        total_info = ''
        self.num_cnt = 0

        while self.num_cnt < self.data_byte:
            recv_msg, recv_addr = self.udp_socket.recvfrom(1024)
            info = recv_msg.hex()
            bytes_info = bytes.fromhex(info)
            rx_num_once = len(bytes_info)
            self.num_cnt += rx_num_once
            total_bytes_info += bytes_info
            total_info += info

        self.int_info = []
        for i in range(0, len(total_bytes_info), 4):
            four_bytes = total_bytes_info[i:i + 4]
            temp_int = struct.unpack('<I', bytes(four_bytes))[0]
            self.int_info.append(temp_int)

        self.plot_signal_data_info.emit(self.int_info)
        self.num_cnt = 0
```

- **功能：**持续监听 UDP 数据，接收并处理数据。
- **说明：**

- 接收数据并累加，直到达到指定的字节数。
- 将接收到的字节数据转换为整数列表。
- 发送处理后的数据到 GUI。

6. UDP 客户端启动

```
def udp_client_start(self, ip: str, port: int) -> None:
    """
    确认UDP客户端的ip及地址
    """
    self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    self.udp_socket.bind(("", 9000))
    self.send_address = (ip, port)
    msg = "UDP客户端已启动\n"
    self.udp_signal_write_msg.emit(msg)
```

- **功能：**启动 UDP 客户端，绑定本地端口并设置目标地址。
- **说明：**创建 UDP 套接字并绑定到本地端口，设置目标 IP 和端口。

7. UDP 数据发送

```
def udp_send(self, send_info: str) -> None:
    """
    功能函数，用于UDP客户端发送消息
    """
    try:
        send_info_encoded = send_info.encode("utf-8")
        self.udp_socket.sendto(send_info_encoded, self.send_address)
        msg = "UDP客户端已发送"
        self.udp_signal_write_msg.emit(msg)
    except Exception as ret:
        msg = "发送失败\n"
        self.udp_signal_write_msg.emit(msg)
```

- **功能：**发送数据到目标地址。
- **说明：**将字符串编码为字节并发送到目标地址。

8. 关闭 UDP 连接

```
def udp_close(self) -> None:
    """
    功能函数，关闭网络连接的方法
    """
    if self.link_flag == self.ServerUDP:
        try:
            self.udp_socket.close()
            msg = "已断开网络\n"
            self.udp_signal_write_msg.emit(msg)
        except Exception as ret:
            pass
        try:
            StopThreading.stop_thread(self.sever_th)
        except Exception:
```

pass

- **功能：**关闭 UDP 套接字并停止监听线程。
- **说明：**关闭套接字并停止线程，确保资源释放。